AWS REST API Lab Part 2: Add Methods and Deploy to a Stage

To Begin with the Lab

Summary of the Lab

In this part of the **REST API** lab, the three HTTP methods were attached to the existing resources in **API Gateway**. The lab reused the **/user** and **/user/{userId}** resources created earlier, and each method was mapped to one of the three **AWS Lambda** functions from the backend: POST for creating a user, GET for reading a user, and DELETE for deleting a user. Unlike the earlier **HTTP API** lab, this stage required manual deployment to a stage before the API could be invoked. A new stage named **dev** was created and the API was deployed to it. After deployment, API Gateway generated the invoke URL that will be used for testing in the next part of the lab.

Understanding REST API Methods and Stages

In **API Gateway REST API**, resources define the URL paths and methods define the actions that happen on those paths. This exists so you can separately control GET, POST, and DELETE behavior for each resource. In this lab, the /user resource handles user collection operations, while /user/{userId} handles specific user operations. A REST API also requires a stage before it can be used, which is why the API must be deployed to a stage such as dev to generate an invoke URL.

Example:

A frontend app sends a POST request to /user to create a new user, a GET request to /user/101 to fetch user 101, and a DELETE request to /user/101 to remove that user. API Gateway routes each method to the correct Lambda function. The deployed stage makes those endpoints available through a real invoke URL.

Lab Steps

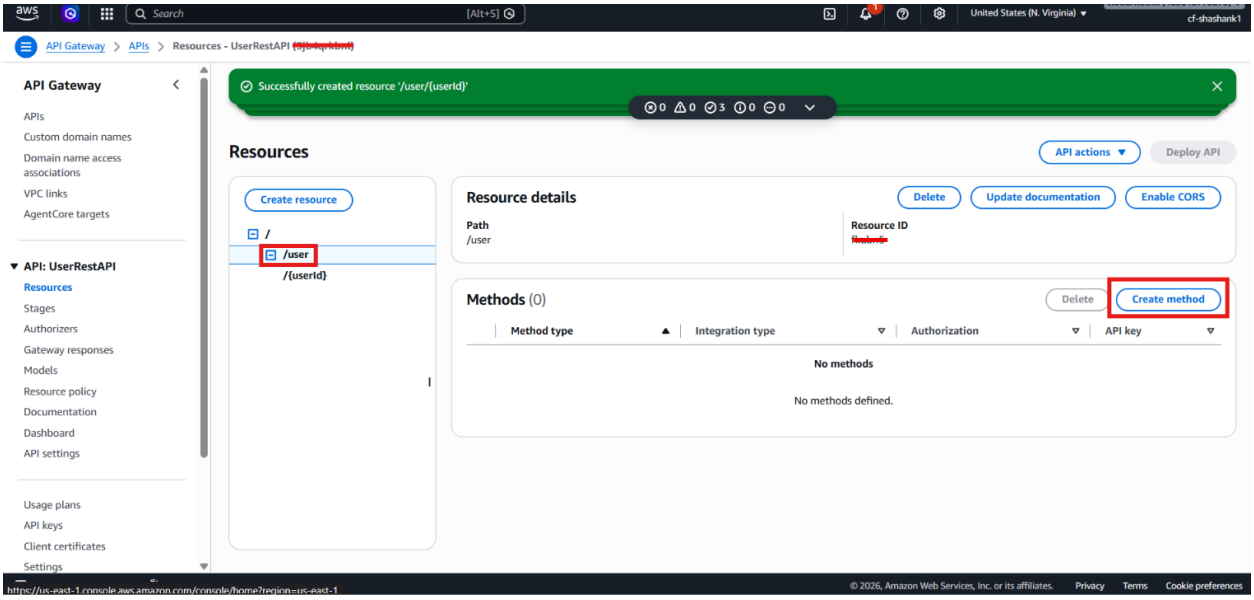
Step 1 – Open the REST API

- Sign in to the AWS Management Console.
- Open **API Gateway**.
- Select the REST API created in the previous part.
- Open the **Resources** section.

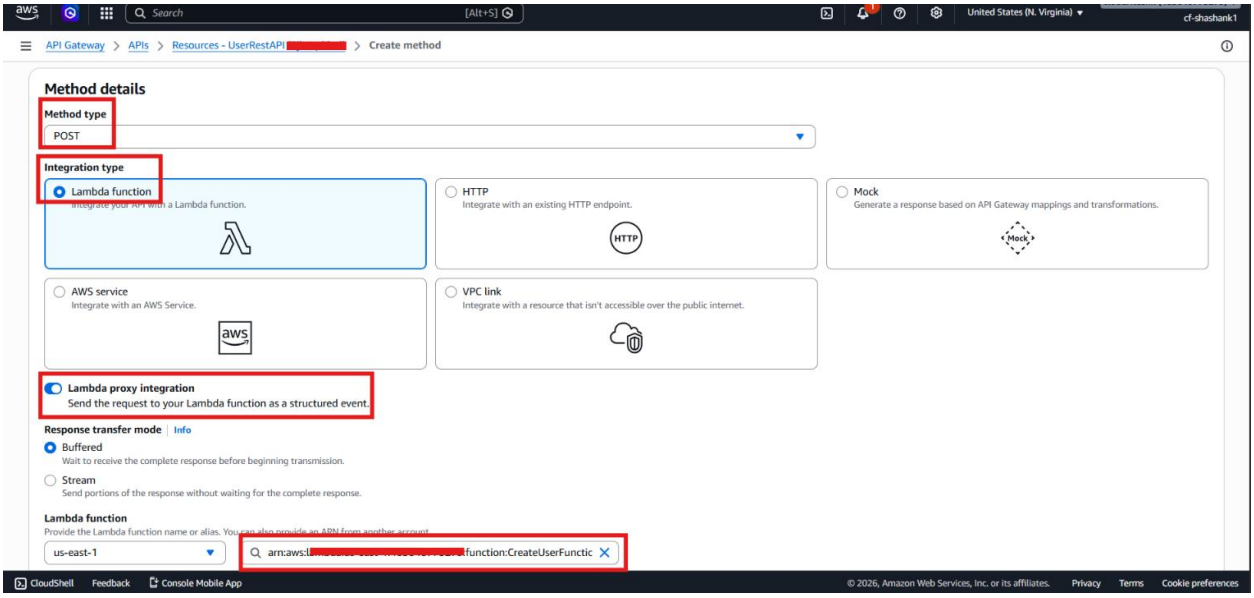
Step 2 – Create the POST Method

- Select the **/user** resource.

- Click **Create Method**.

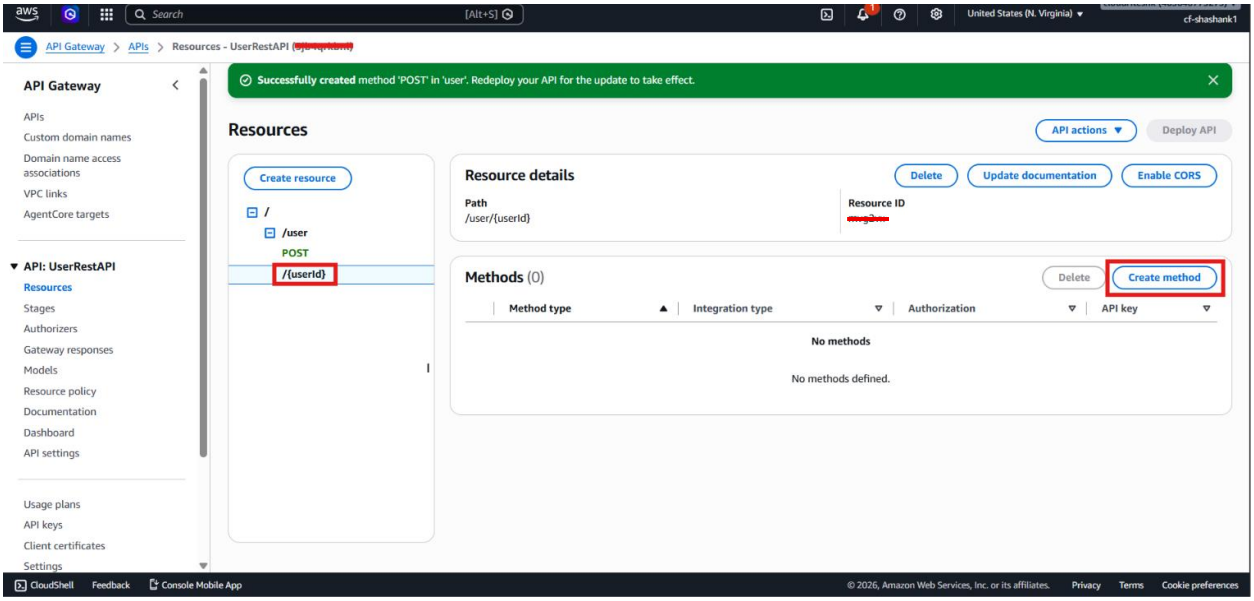


- Choose **POST**.
- Enable **Lambda Proxy Integration**.
- Keep the response transfer mode as **Buffered**.
- Select the Lambda function: CreateUserFunction
- Click **Create Method**.

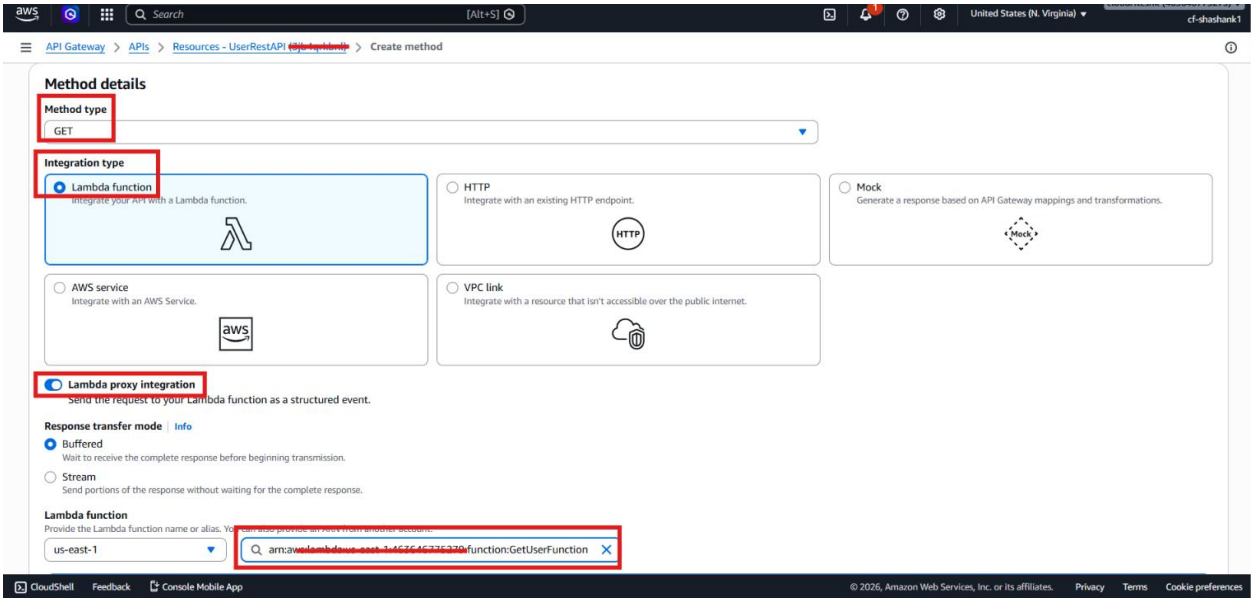


Step 3 – Create the GET Method

- Select the **/user/{userId}** resource.
- Click **Create Method**.

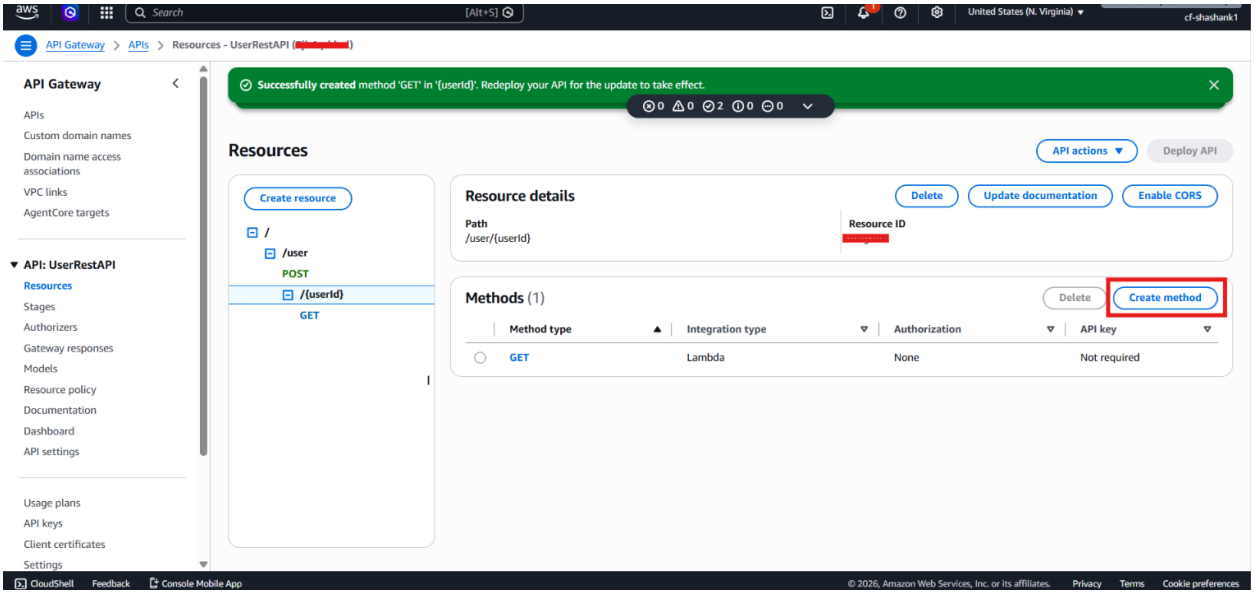


- Choose **GET**.
- Enable **Lambda Proxy Integration**.
- Keep the response transfer mode as **Buffered**.
- Select the Lambda function: getUserFunction
- Click **Create Method**.

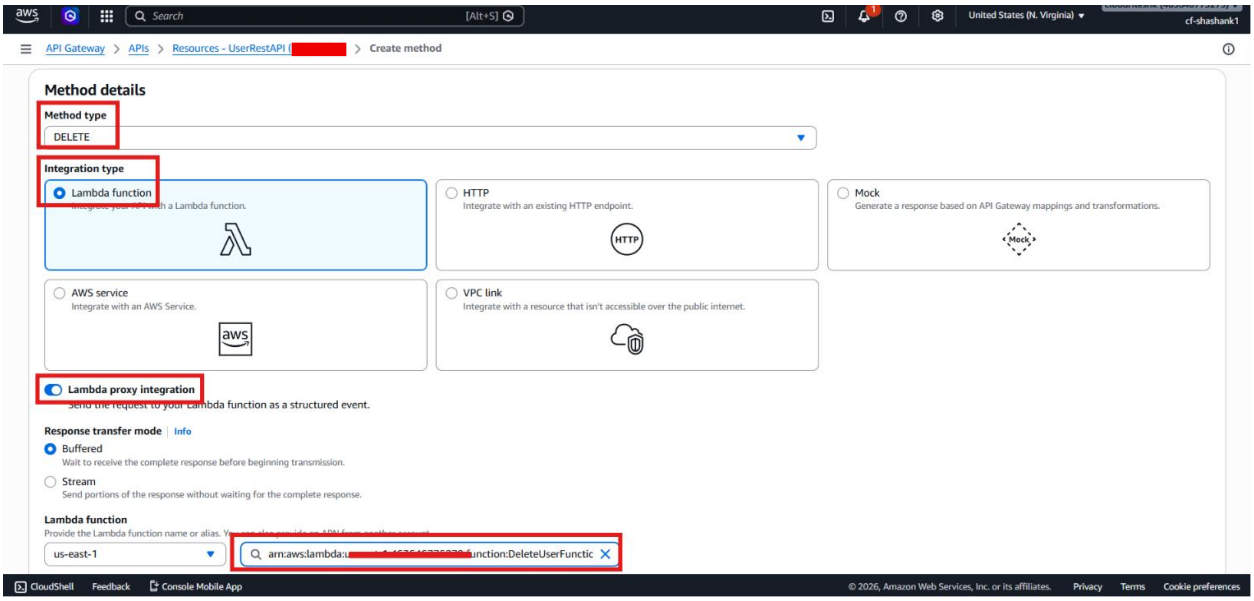


Step 4 – Create the DELETE Method

- Select the **/user/{userId}** resource.
- Click **Create Method**.



- Choose **DELETE**.
- Enable **Lambda Proxy Integration**.
- Keep the response transfer mode as **Buffered**.
- Select the Lambda function: deleteUserFunction
- Click **Create Method**.

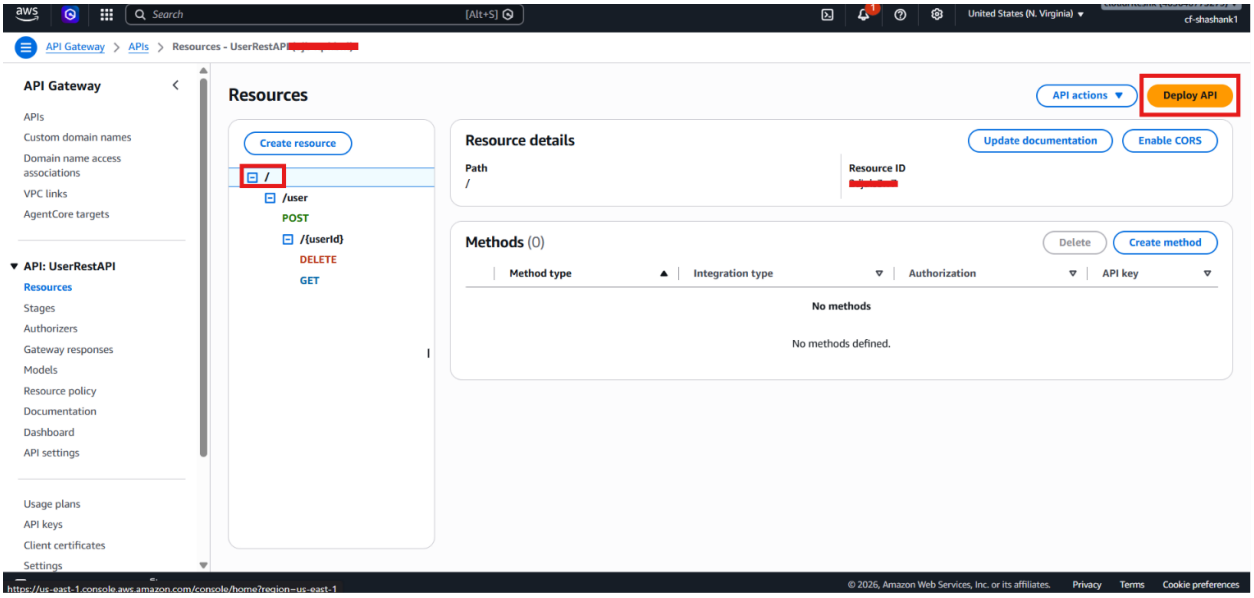


Step 5 – Review the Method Mapping

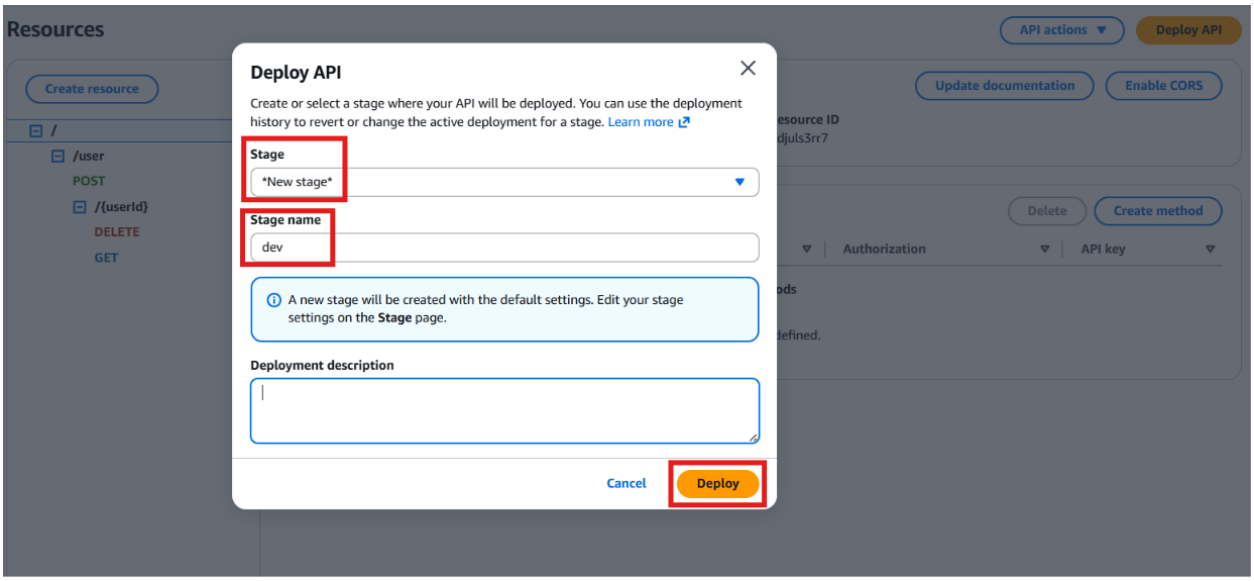
- Confirm the method-to-Lambda mapping:
  - **POST** /user → CreateUserFunction
  - **GET** /user/{userId} → getUserFunction
  - **DELETE** /user/{userId} → deleteUserFunction
- Wait for API Gateway to finish updating the API.

Step 6 – Deploy the API

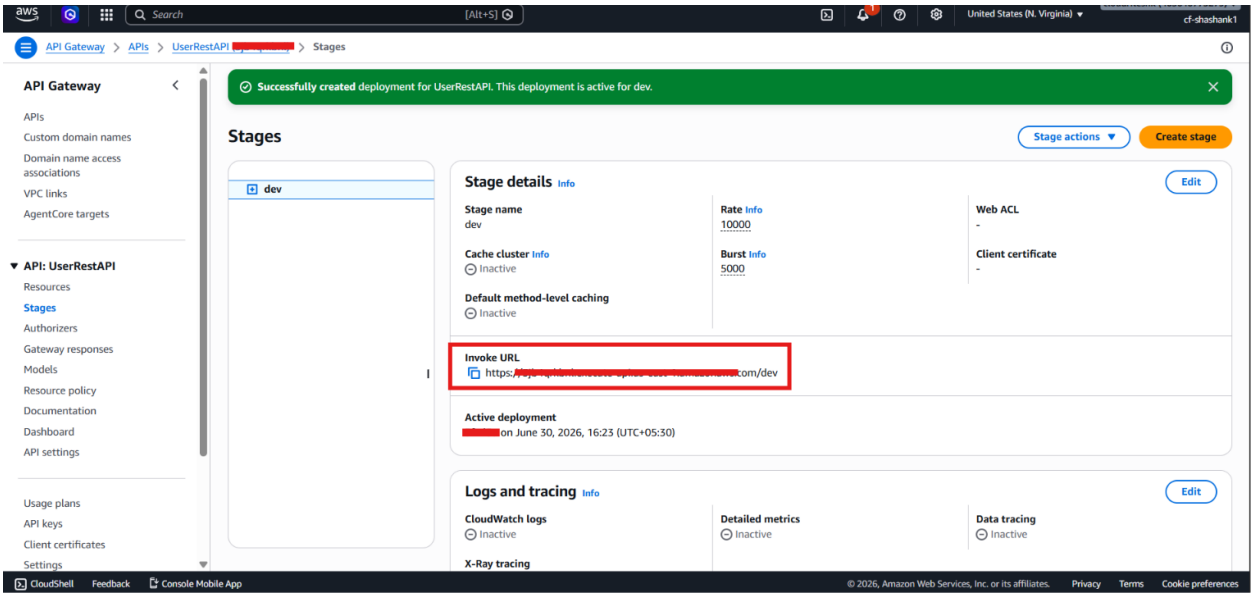
- Click **Deploy API**.



- Choose **New Stage**.
- Enter the stage name: dev
- Click **Deploy**.



- Wait for the deployment to complete.
- Copy the generated **Invoke URL**.



- Confirm that the API is now available through the deployed stage.

Step 7 – Prepare for Testing

- Verify that all three methods are active.
- Confirm that the **dev** stage exists.
- Save the invoke URL for use in the next part of the lab.
- Prepare to test the API in **Postman** or another client tool.

Verification

- The three REST API methods were created successfully.
- Each method was connected to the correct **AWS Lambda** function.
- The REST API was deployed to the **dev** stage.
- API Gateway generated an invoke URL.
- The API is ready for end-to-end testing in the final part of the lab.